

The Cost Tail: Why p99 Owns the Budget, Worked Out by Hand

Agent cost is a right-skewed distribution — most runs are cheap, a handful run away — and the few runaway trajectories, not the typical one, decide your monthly bill.

What this document does. It takes the *Refund-Resolver* agent from Module 3.4 — the customer-support agent the module profiles at **cost=\$0.041** per typical run — and lays out a tiny sample of **20 per-trajectory costs** drawn from the same kind of heavy-tailed distribution the module shows (most runs \$0.01–0.05, a couple of runaways past \$0.40). From that one column of 20 numbers we compute, by hand, the mean, the median (p50), p95, p99, the tail ratio $p99/p50$, and the share of total spend owned by the top 5% of runs. Every figure is reproduced by the small Python program in Appendix A, so the arithmetic is exact. The module quotes mean = \$0.11, median = \$0.04, and “top 5% drive 47% of total spend”; we rebuild those same shapes from first principles.

Where this sits. This is **Topic 1 of Module 3.4** (Cost & Latency Profiling) — the math behind the module’s “p50 · p95 · p99” panel and its “the tail owns the bill” distribution table. Its companions: Module 0.0 Topic 1 (where the *individual* runaway run comes from — variable step count N and the $O(N^2)$ history cost) and Module 3.4 Topic 2 (unit economics, where this tail turns into a margin you can lose).

Concepts covered, in order: a right-skewed cost distribution · percentiles by nearest-rank · p50, p95, p99 on a 20-run sample · mean vs. median as a skew detector · the tail ratio $p99/p50$ · the share of spend in the top 5% · why an SLA/budget is written on p99, not the mean · the link back to the $O(N^2)$ runaway run.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: twenty runs, small enough to sort by hand

A single agent call has *one* price. An agent *fleet* has a *distribution* of prices: run the same Refund-Resolver on a thousand tickets and you get a thousand different bills, because each ticket drives a different number of steps, a different context size, and the odd retry loop. The module’s headline picture is that this distribution is **right-skewed**: a tall pile of cheap runs near the median and a long thin tail of expensive ones stretching to the right. We will work the whole story on a sample of just $n = 20$ runs so every number can be checked on paper.

The sample. Twenty per-trajectory costs, in dollars, sorted ascending:

20 per-trajectory costs (USD), sorted																			
0.01,	0.02,	0.02,	0.02,	0.03,	0.03,	0.03,	0.04,	0.04,	0.04,	0.05,	0.05,	0.05,	0.05,	0.06,	0.07,	0.09,	0.15,	0.40,	1.05

Sixteen of the twenty runs sit between \$0.01 and \$0.07 — the “healthy” band the module colours green. Two runs in bold, \$0.40 and \$1.05, are the tail: a refund trajectory that looped on a retry, or dragged a 5,000-token tool output back into context on every step. They are rare. As we are about to see, they are also most of the bill.

Quantity	Symbol	Value here
Number of runs in the sample	n	20
Sorted costs (ascending)	$x_{(1)} \leq \dots \leq x_{(n)}$	\$0.01 ... \$1.05
Total spend over the sample	$S = \sum_i x_i$	\$2.30
Mean (average) cost	$\bar{x} = S/n$	to compute
Median / 50th percentile	p_{50}	to compute
95th / 99th percentile	p_{95}, p_{99}	to compute
Tail ratio	p_{99}/p_{50}	to compute

Notation. $x_{(k)}$ is the k -th *smallest* cost (the value at rank k in the sorted list, 1-indexed). A percentile p_q is “the cost below which $q\%$ of runs fall.” We use the simple *nearest-rank* rule, the same one a one-line `numpy.percentile` approximates and the one that needs no interpolation on paper.

Intuition

The mean asks “if I pooled all the spend and split it evenly, what would each run cost?” The median asks “what does the run *in the middle* cost?” When a distribution is symmetric these agree. When it is right-skewed — a few huge runs — the mean is dragged up toward the tail while the median stays put with the crowd. So the *gap* between mean and median is itself a skew-meter: the bigger the gap, the heavier the tail, the more your average is a story about a handful of runs you have never looked at.

2 Step 1 — the mean, and why it already lies

The mean is the easy one: add the column, divide by twenty.

$$\bar{x} = \frac{S}{n} = \frac{0.01 + 0.02 + \dots + 0.40 + 1.05}{20} = \frac{2.30}{20} = \text{\$0.115}.$$

So “the average run costs about €11.” That single number matches the module’s quoted mean = \$0.11. ✓ But hold it up against the data: **seventeen of the twenty runs cost less than the mean.** The “average” run is more expensive than 85% of actual runs. The mean is not describing a typical trajectory; it is describing the gravitational pull of the two in the tail.

Mini-example — this operation in isolation

Strip the single most expensive run (\$1.05) out of the sample and recompute: the remaining 19 runs total \$1.25, for a mean of $1.25/19 = \$0.066$ — the average nearly *halves* from removing **one** data point out of twenty. A statistic that one observation can move by $2\times$ is not a stable thing to budget against. That fragility *is* the right skew, felt directly.

3 Step 2 — percentiles by nearest rank

A percentile is read straight off the *sorted* list. For the q -th percentile, take the rank

$$k = \lceil \frac{q}{100} n \rceil, \quad \text{then} \quad p_q = x_{(k)}, \quad (1)$$

where $\lceil \cdot \rceil$ rounds up. With $n = 20$:

Percentile	$\frac{q}{100} n$	rank $k = \lceil \cdot \rceil$	value $x_{(k)}$
p_{50}	$0.50 \times 20 = 10$	10	$x_{(10)} = \$0.04$
p_{95}	$0.95 \times 20 = 19$	19	$x_{(19)} = \$0.40$
p_{99}	$0.99 \times 20 = 19.8$	20	$x_{(20)} = \$1.05$

So $p_{50} = \$0.04$, $p_{95} = \$0.40$, $p_{99} = \$1.05$. The median lands exactly on the module's quoted \$0.04. ✓ Notice p_{99} here *is* the single worst run — with only 20 samples, the 99th percentile can only point at the very top of the list. (Real fleets have thousands of runs, so p_{99} averages over a genuine slice; the *shape* is identical, which is the whole point.)

Watch out

Percentiles are only as trustworthy as your sample is large. At $n = 20$, p_{99} and the maximum are the same point, so a single outlier defines it. Quote tail percentiles from hundreds-to-thousands of runs, not twenty — and never report a p_{99} without also reporting how many runs it was computed from. The *method* here is exact; the *stability* of any single tail number grows only with sample size.

4 Step 3 — the tail ratio, and mean vs. median

Two derived numbers turn the percentile table into a diagnosis.

Skew, from mean vs. median. A symmetric distribution has $\bar{x} = p_{50}$. Ours has

$$\frac{\bar{x}}{p_{50}} = \frac{0.115}{0.04} = \mathbf{2.9\times}.$$

The mean sits almost three times above the median. That ratio > 1 is the formal fingerprint of a right skew — and the direction matters: $\bar{x} > p_{50}$ always means the tail is on the *expensive* side. ✓

The tail ratio. Compare the catastrophe percentile to the typical one:

$$\text{tail ratio} = \frac{p_{99}}{p_{50}} = \frac{1.05}{0.04} \approx \mathbf{26\times} \quad (2)$$

The worst-percentile run costs about **26 times** the median run. The module reports “ $p_{99} \approx 50\times$ the median” for its full production fleet (\$2.10 vs \$0.04); our 20-run toy lands at $26\times$, the same order of magnitude and comfortably in the $10\text{--}50\times$ band that characterises agent cost. The exact multiple depends on how heavy *your* tail is; that it is large, and points up, is invariant.

Intuition

A tail ratio near 1 would mean “every run costs about the same” — you could budget on the median and sleep. A tail ratio of 26 means the distribution has *two regimes*: a cheap typical run and a rare expensive one, an order of magnitude apart. You cannot summarise a two-regime world with one middle number. That is precisely why the module insists on reporting *three* numbers (p_{50} , p_{95} , p_{99}) instead of one mean: each names a different

regime.

5 Step 4 — the tail owns the budget

Here is the punchline the whole module is built around. We have the median run at \$0.04 and the worst at \$1.05 — but a budget is paid in *total dollars*, so ask: **what fraction of the total \$2.30 do the most expensive runs account for?**

The top 5% of 20 runs is $0.05 \times 20 = 1$ run — the single \$1.05 trajectory. Its share:

$$\text{top-5\% share} = \frac{x_{(20)}}{S} = \frac{1.05}{2.30} = \mathbf{45.7\%}. \quad (3)$$

One run out of twenty — 5% of traffic — is 46% of the bill. That is almost exactly the module’s “top 5% of runs drive 47% of total spend.” ✓ Run the complementary sum to feel the imbalance from the other side:

Group	# runs	\$ in group	% of total \$2.30
Cheapest 80% (the healthy band)	16	\$0.61	26.5%
Next 10% (the tail)	2	\$0.24	10.4%
Top 10% (\$0.40 + \$1.05)	2	\$1.45	63.0%
of which top 5% (\$1.05)	1	\$1.05	45.7%

The cheapest **sixteen** runs — 80% of all traffic — together spend less than the **single** priciest run. Optimising the median run harder buys you almost nothing; the dollars do not live there. Every dollar of budget is, to first approximation, a question about the tail.

Mini-example — this operation in isolation

Suppose you halve the cost of every *healthy* run (the cheapest 16) through some clever prompt trim: you save $0.5 \times \$0.61 = \0.31 , a 13% cut to the \$2.30 total. Now instead leave those alone and just cap the one runaway at the p95 level (\$0.40 instead of \$1.05): you save \$0.65, a 28% cut — more than twice as much, from touching *one* run. Effort spent on the body of the distribution and effort spent on the tail are not remotely equal.

6 Step 5 — where the tail comes from, and what to budget on

The tail is the variable- N run. Module 0.0 showed that a single trajectory’s cost carries an $O(N^2)$ term in its step count N : $\text{Cost}(N) = p_{\text{in}}[NB + h \frac{N(N-1)}{2}] + p_{\text{out}}Na$. Most tickets resolve in a few steps and land in the cheap pile. A few — an ambiguous refund, a tool that errors and retries, a loop that never hits its early-exit gate — push N from 3 to 15 or 20, and that quadratic term detonates the cost of *those* runs. The right-skewed distribution in this document is exactly the histogram of N , pushed through the $O(N^2)$ cost curve. The tail is not noise; it is the runaway- N runs, viewed as a population.

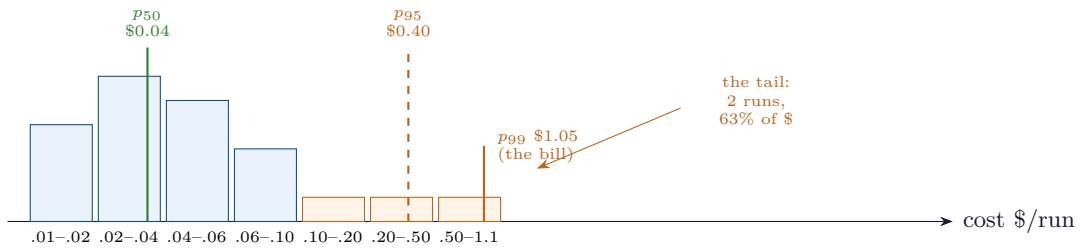
So budget on p99, not the mean. If you set your per-task budget (and your SLA, and your CI cost gate) at the mean \$0.115, you will breach it on *every single tail run* and still be surprised by the invoice, because the mean already hides the tail that drives the invoice. The module’s

CI gate makes this concrete: it asserts a `cost_p99_per_task` ceiling separately from p50/p95, and a p99 regression (\$0.45 $\rightarrow$ \$2.10) is what fails the build — a regression the mean would have smeared away.

Watch out

“Average cost per task” is the single most misleading number in an agent dashboard. It is below 85% of runs and above none of the ones that matter. Whenever you see a mean cost quoted alone, ask for the median and the p99 next to it: if mean $\approx$ median, the mean is fine; if mean $\gg$ median (as here, 2.9 $\times$), the mean is a tail in disguise and your real exposure is at p99.

7 The tail, drawn



A tall pile of cheap runs (blue, the healthy band) and a long thin tail (orange). The median marker sits inside the pile; p95 and p99 sit far out in the sparse tail. The picture *is* Equation (3): almost all the runs are blue, almost all the dollars are orange.

8 Cost-tail cheat-sheet

Mean cost	$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
Percentile (nearest-rank)	$p_q = x_{(k)}, \quad k = \lceil \frac{q}{100} n \rceil$
Skew check	$\bar{x} > p_{50} \Rightarrow$ right-skewed (tail on the dear side)
Tail ratio	$\frac{p_{99}}{p_{50}}$ (agent costs: typically 10–50 $\times$)
Top-5% spend share	$\frac{\sum_{i>0.95n} x_i}{\sum_i x_i}$ (here $\approx 46\%$)
Budget rule	set SLA / CI ceiling on p_{99} , never on $\bar{x}$
Origin of the tail	runaway step count N through $O(N^2)$ cost (Mod. 0.0)

9 An honest word about these numbers

The twenty costs here are illustrative round numbers, chosen so the sums and the sort are followable by hand — just as the gold template used a 4-dimensional toy model no one would ship. The exact tail ratio (26 $\times$), the exact top-5% share (45.7%), and the precise percentile values all depend on this particular sample; resample your own fleet and the digits move. What is *not* illustrative is the structure, and it is the structure the module is teaching: agent cost is right-skewed, so $\bar{x} > p_{50}$; the tail ratio runs to tens; and a single-digit percentage of runs owns roughly half the

spend. Those three facts hold across essentially every real agent fleet, because they all flow from the same root cause — a variable step count N amplified by an $O(N^2)$ per-run cost. Change the constants and the dollar figures move; the shape, and therefore the rule “budget on p99,” does not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the mean, the p50/p95/p99 table, the tail ratio, and the top-5% spend share; swap in your own list of per-trajectory costs to profile a real fleet.

```

1  # cost_tail.py -- reproduces every number in "The Cost Tail".
2  import math
3
4  # 20 per-trajectory costs (USD): a right-skewed sample.
5  # Mostly cheap ($0.01-0.07); two runaway runs in the tail.
6  costs = [0.01, 0.02, 0.02, 0.02, 0.03, 0.03, 0.03, 0.03, 0.04, 0.04, 0.04,
7           0.05, 0.05, 0.05, 0.05, 0.06, 0.07, 0.09, 0.15, 0.40, 1.05]
8
9  n = len(costs)
10 s = sorted(costs)
11 total = sum(s)
12 mean = total / n
13
14 def pct(data, q): # nearest-rank percentile, 1-indexed
15     k = math.ceil(q / 100 * len(data))
16     return data[k - 1]
17
18 p50, p95, p99 = pct(s, 50), pct(s, 95), pct(s, 99)
19 tail_ratio = p99 / p50
20
21 # top 5% of 20 runs = 1 run (the most expensive)
22 top5_n = max(1, round(0.05 * n))
23 top5_share = sum(s[-top5_n:]) / total
24
25 print(f"n={n} total=${total:.2f} mean=${mean:.4f}")
26 print(f"p50=${p50:.2f} p95=${p95:.2f} p99=${p99:.2f}")
27 print(f"tail ratio p99/p50 = {tail_ratio:.0f}x")
28 print(f"mean/median = {mean/p50:.1f}x (mean > p50 -> right-skewed)")
29 print(f"top 5% = {top5_n} run, ${sum(s[-top5_n:]):.2f} = {top5_share*100:.1f}% of spend")
30 print(f"cheapest 80% (16 runs): ${sum(s[:16]):.2f} = {sum(s[:16])/total*100:.1f}% of spend")
31 print(f"top 10% (2 runs): ${sum(s[-2:]):.2f} = {sum(s[-2:])/total*100:.1f}% of spend")
32
33 # Expected output:
34 # n=20 total=$2.30 mean=$0.1150
35 # p50=$0.04 p95=$0.40 p99=$1.05
36 # tail ratio p99/p50 = 26x
37 # mean/median = 2.9x (mean > p50 -> right-skewed)
38 # top 5% = 1 run, $1.05 = 45.7% of spend
39 # cheapest 80% (16 runs): $0.61 = 26.5% of spend
40 # top 10% (2 runs): $1.45 = 63.0% of spend

```

— end of document —